In [9]:

```python
import pandas as pd
from pathlib import Path
import os
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import gaussian_kde, skew, kurtosis, norm
from scipy.stats import pearsonr
import statsmodels.api as sm
import seaborn as sns
from statsmodels.nonparametric.smoothers_lowess import lowess
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
from linearmodels.panel import RandomEffects
from linearmodels.panel import PanelOLS
import statsmodels.api as sm
from linearmodels.iv import IV2SLS
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from scipy.stats import chi2
```

In [4]:

```python
# loading in my data frame classic first steps same as the other part in part 1

final_data_path = Path("/Users/lorenzouberti/Desktop/Panel-Data-Project-Financial-Econometrics/Data")
df_Final = pd.read_excel(final_data_path / "final_panel_dataset.xlsx")

# In the study as one of teh explanatory varibles they create a 5 year lagged GDP per capita variable.
# Therefore, prior to plotting the distributions, we create this variable in the final data set.

df_Final = df_Final.sort_values(["country", "year"])
df_Final = df_Final[['country', 'year', 'gdppc', 'labour_productivity', 'cpi', 'trade_openness', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']]
df_Final = df_Final.sort_values(["country", "year"])

# Logging GDP per capita first
df_Final['ln_gdppc'] = np.log(df_Final['gdppc'])
df_Final["ln_gdppc_lag5"] = df_Final.groupby("country")["ln_gdppc"].shift(5)
# Creating the 5 year lagged GDP per capita growth variable
df_Final["ln_gdp_growth_5y"] = df_Final["ln_gdppc"] - df_Final["ln_gdppc_lag5"]

# Doing of rthe alternative dependent variable: labour productivity growth
df_Final['ln_labour_productivity'] = np.log(df_Final['labour_productivity'])
df_Final["ln_labour_productivity_lag5"] = df_Final.groupby("country")["ln_labour_productivity"].shift(5)
df_Final["ln_labour_productivity_growth_5y"] = df_Final["ln_labour_productivity"] - df_Final["ln_labour_productivity_lag5"]

# taking teh logarithm of the other variables as well
df_Final['ln_cpi'] = np.log(df_Final['cpi'])
df_Final['ln_trade_openness'] = np.log(df_Final['trade_openness'])


variables = [ 'ln_gdppc','ln_labour_productivity', 'ln_cpi', 'ln_trade_openness', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp', "ln_gdp_growth_5y", "ln_labour_productivity_growth_5y" ]

for var in variables:   # Between transformation
    df_Final[f'{var}_between'] = df_Final.groupby('country')[var].transform('mean')

    # One-way Within transformation
    df_Final[f'{var}_within'] = df_Final[var] - df_Final.groupby('country')[var].transform('mean')

rows = []

# Defining my Dependent and Explanatory Variables
Dependent_Variables_Growth = ['ln_gdp_growth_5y','ln_labour_productivity_growth_5y' ]
Explanatory_Variables = ['ln_gdppc_lag5', 'ln_labour_productivity_lag5', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp']
Control_Variables = ['ln_cpi', 'ln_trade_openness']

individuals = df_Final['country']
time = df_Final['year']
dep_var_col = 'ln_gdp_growth_5y'
exp_var_col = 'liquid_liabilities'
cols_to_keep = ["country", "year", dep_var_col, exp_var_col, "ln_cpi", "ln_trade_openness"]
df_Final_defined = df_Final[cols_to_keep].dropna()

df_Final_defined["dep_var_col_mean_i"] = df_Final_defined.groupby(individuals)[dep_var_col].transform("mean")
df_Final_defined["exp_var_col_mean_i"] = df_Final_defined.groupby(individuals)[exp_var_col].transform("mean")

df_Final_defined["dep_var_col_within"] = df_Final_defined[dep_var_col] - df_Final_defined['dep_var_col_mean_i']
df_Final_defined["exp_var_col_within"] = df_Final_defined[exp_var_col] - df_Final_defined['exp_var_col_mean_i']

between_df_Final_defined = (
    df_Final_defined
    .groupby("country")
    .agg(
        dep_var_col_between=(dep_var_col, 'mean'),
        exp_var_col_between=(exp_var_col, 'mean')
    )
    .reset_index()
)

# TWFE transformation
df_twfe = df_Final.copy()


id_col = "country"
time_col = "year"
cols_twfe = ["liquid_liabilities","mcap_gdp","bank_loans_gdp","ln_gdp_growth_5y","ln_labour_productivity_growth_5y", "ln_cpi", "ln_trade_openness"]
for col in cols_twfe:
    mi = df_twfe.groupby(id_col)[col].transform("mean")
    mt = df_twfe.groupby(time_col)[col].transform("mean")
    mall = df_twfe[col].mean()
    df_twfe[col + "_twfe"] = df_twfe[col] - mi - mt + mall

# all the the data witgh the respecive data frame is on df_TWFE
```

Question 20

In [5]:

```python
id_col = "country"
time_col = "year"

dep_var = "ln_gdp_growth_5y"
x_vars = ["ln_gdppc_lag5", "liquid_liabilities", "mcap_gdp", "bank_loans_gdp"]
control_vars = ["ln_cpi", "ln_trade_openness"]

VIF_cutoff = 10.0
cluster = True

# variance inflation factor calculation
def calculate_vif(df, features):
    X = df[features]
    X = sm.add_constant(X, has_constant="add")
    vals = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
    cols = list(X.columns)
    vif_series = pd.Series(vals, index=cols, name="VIF")
    if "const" in vif_series.index:
        vif_series = vif_series.drop("const")
    else:
        vif_series = vif_series.iloc[1:] if len(vif_series) > 0 else vif_series
    return vif_series

def drop_high_vif(X: pd.DataFrame, vif_cut: float, keep: set = None, verbose: bool = True):
    if keep is None:
        keep = set()

    Xw = X.copy()

    while True:
        v = calculate_vif(Xw, list(Xw.columns))
        droppable = [c for c in Xw.columns if c not in keep]
        if not droppable:
            break
        v_droppable = v.reindex(droppable).astype(float)
        worst = v_droppable.idxmax()
        worst_val = v_droppable.loc[worst]
        if np.isfinite(worst_val) and worst_val > vif_cut:
            if verbose:
                print(f"    [VIF] dropping '{worst}' (VIF={worst_val:.2f} > {vif_cut})")
            Xw = Xw.drop(columns=[worst])
        else:
            break
    return Xw, calculate_vif(Xw, list(Xw.columns))


def fit_ols(y: pd.Series, X: pd.DataFrame, clusters: pd.Series = None, add_const: bool = False):
    X_use = sm.add_constant(X, has_constant="add") if add_const else X
    model = sm.OLS(y.astype(float).values, X_use.astype(float).values)
    if clusters is not None:
        res = model.fit(cov_type="cluster", cov_kwds={"groups": clusters.values})
    else:
        res = model.fit()
    res.X_names = list(X_use.columns)
    return res


def short_print(title: str, res):
    print("\n" + "="*80)
    print(title)
    print("-"*80)
    params = pd.Series(res.params, index=res.X_names)
    print(params)
    print(f"\n nobs={int(res.nobs)}  R2={getattr(res,'rsquared',np.nan):.4f}")
    print("="*80)


def fisher_test_fe(df_levels: pd.DataFrame, y: str, X_cols: list):
    d = df_levels[[id_col, y] + X_cols].dropna().copy()
    for c in [y] + X_cols:
        d[c] = pd.to_numeric(d[c], errors="coerce")
    d = d.dropna(subset=[y] + X_cols)
    X_r = sm.add_constant(d[X_cols], has_constant="add").astype(float)
    yv = d[y].astype(float).values
    res_r = sm.OLS(yv, X_r.values).fit()
    dums = pd.get_dummies(d[id_col], drop_first=True, prefix="fe", dtype=float)
    X_u = pd.concat([X_r, dums], axis=1).astype(float)
    res_u = sm.OLS(yv, X_u.values).fit()
    q = dums.shape[1]
    n = len(d)
    k_u = X_u.shape[1]
    F = ((res_r.ssr - res_u.ssr) / q) / (res_u.ssr / (n - k_u))
    p = 1 - stats.f.cdf(F, q, n - k_u)
    return {"F": float(F), "df1": int(q), "df2": int(n - k_u), "p": float(p)}

def ancova_poolability_test(df_levels: pd.DataFrame, y: str, X_cols: list, add_time_fe: bool = False):
    d = df_levels[[id_col, time_col, y] + X_cols].dropna().copy()
    for c in [y] + X_cols:
        d[c] = pd.to_numeric(d[c], errors="coerce")
    d[time_col] = pd.to_numeric(d[time_col], errors="coerce")
    d = d.dropna(subset=[time_col, y] + X_cols)
    Xbase = sm.add_constant(d[X_cols], has_constant="add").astype(float)
    fe = pd.get_dummies(d[id_col], drop_first=True, prefix="fe", dtype=float)
    parts = [Xbase, fe]
    if add_time_fe:
        te = pd.get_dummies(d[time_col].astype(int), drop_first=True, prefix="t", dtype=float)
        parts.append(te)
    X_base = pd.concat(parts, axis=1).astype(float)
    inter_blocks = []
    for x in X_cols:
        inter_blocks.append(fe.mul(d[x].astype(float), axis=0))
    X_inter = pd.concat(inter_blocks, axis=1).astype(float)
    X_inter.columns = [f"{c}#c.{x}" for x in X_cols for c in fe.columns]

    X_unres = pd.concat([X_base, X_inter], axis=1).astype(float)

    yv = d[y].astype(float).values
    res_base = sm.OLS(yv, X_base.values).fit()
    res_unres = sm.OLS(yv, X_unres.values).fit()

    q = X_inter.shape[1]
    n = len(d)
    k_u = X_unres.shape[1]
    F = ((res_base.ssr - res_unres.ssr) / q) / (res_unres.ssr / (n - k_u))
    p = 1 - stats.f.cdf(F, q, n - k_u)
    return {"F": float(F), "df1": int(q), "df2": int(n - k_u), "p": float(p), "time_fe": add_time_fe}



def chow_group_test(df_levels: pd.DataFrame, y: str, X_cols: list, group_col: str):
    d = df_levels[[y, group_col] + X_cols].dropna().copy()
    g = d[group_col].astype(float)

    X0 = sm.add_constant(d[X_cols], has_constant="add")
    inter = pd.concat([(g * d[x]).rename(f"{group_col}*{x}") for x in X_cols], axis=1)
    X = pd.concat([X0, g.rename(group_col), inter], axis=1)

    res = sm.OLS(d[y].values, X.values).fit()

    k0 = X0.shape[1] + 1  # const + X + g
    R = np.zeros((len(X_cols), X.shape[1]))
    for i in range(len(X_cols)):
        R[i, k0 + i] = 1.0
    wald = res.wald_test(R)
    return {"chi2": float(wald.statistic), "df": len(X_cols), "p": float(wald.pvalue)}

def chow_time_break(df_levels: pd.DataFrame, y: str, X_cols: list, break_year: int):
    d = df_levels[[time_col, y] + X_cols].dropna().copy()
    X_full = sm.add_constant(d[X_cols], has_constant="add")
    res_full = sm.OLS(d[y].values, X_full.values).fit()

    d1 = d[d[time_col] < break_year]
    d2 = d[d[time_col] >= break_year]
    X1 = sm.add_constant(d1[X_cols], has_constant="add")
    X2 = sm.add_constant(d2[X_cols], has_constant="add")
    res1 = sm.OLS(d1[y].values, X1.values).fit()
    res2 = sm.OLS(d2[y].values, X2.values).fit()

    SSR_pooled = res_full.ssr
    SSR_split  = res1.ssr + res2.ssr
    k = X_full.shape[1]
    n1, n2 = len(d1), len(d2)

    F = ((SSR_pooled - SSR_split) / k) / (SSR_split / (n1 + n2 - 2*k))
    p = 1 - stats.f.cdf(F, k, (n1 + n2 - 2*k))
    return {"F": F, "df1": k, "df2": (n1 + n2 - 2*k), "p": p, "break_year": break_year}

if 'X_ALL' not in globals():
    try:
        X_ALL = Explanatory_Variables + Control_Variables
    except NameError:
        X_ALL = x_vars + control_vars

else:
    try:
        for c in Control_Variables:
            if c not in X_ALL:
                X_ALL.append(c)
    except NameError:
        for c in control_vars:
            if c not in X_ALL:
                X_ALL.append(c)

df_levels = df_Final[[id_col, time_col, dep_var] + X_ALL].dropna().copy()
ID_COL = id_col
T_COL = time_col
DEP = dep_var
VIF_CUT = VIF_cutoff
CLUSTER = cluster
if 'GROUP_COL' not in globals():
    GROUP_COL = None
if 'BREAK_YEAR' not in globals():
    BREAK_YEAR = None

print("\n>>> Fisher FE test (are country effects jointly significant?)")
print(fisher_test_fe(df_levels, DEP, X_ALL))

print("\n>>> ANCOVA / poolability test (common slopes) one-way base")
print(ancova_poolability_test(df_levels, DEP, X_ALL, add_time_fe=False))

print("\n>>> ANCOVA / poolability test (common slopes) two-way base")
print(ancova_poolability_test(df_levels, DEP, X_ALL, add_time_fe=True))

if GROUP_COL is not None:
    print("\n>>> Chow group test (0/1)")
    print(chow_group_test(df_levels, DEP, X_ALL, GROUP_COL))
if BREAK_YEAR is not None:
    print("\n>>> Chow time-break test")
    print(chow_time_break(df_levels, DEP, X_ALL, BREAK_YEAR))


print("\n>>> BETWEEN (BE) regression: OLS on country means (includes constant)")
between = df_levels.groupby(ID_COL)[[DEP] + X_ALL].mean().reset_index()

y_be = between[DEP]
X_be_raw = between[X_ALL]

X_be, vif_be = drop_high_vif(X_be_raw, VIF_CUT, verbose=True)
print("  BE regressors kept:", list(X_be.columns))
print("  VIF (BE):\n", vif_be)

res_be = fit_ols(y_be, X_be, clusters=None, add_const=True)
short_print("BETWEEN (BE)", res_be)

print("\n>>> WITHIN (one-way FE): within-transformed OLS (no constant)")
df_w = df_levels.sort_values([ID_COL, T_COL]).copy()
for c in [DEP] + X_ALL:
    df_w[c + "_within"] = df_w[c] - df_w.groupby(ID_COL)[c].transform("mean")

y_w = df_w[DEP + "_within"]
X_w_raw = df_w[[c + "_within" for c in X_ALL]]

X_w, vif_w = drop_high_vif(X_w_raw, VIF_CUT, verbose=True)
print("  FE(within) regressors kept:", list(X_w.columns))
print("  VIF (FE/within):\n", vif_w)

clusters = df_w[ID_COL] if CLUSTER else None
res_fe = fit_ols(y_w, X_w, clusters=clusters, add_const=False)
short_print("WITHIN / FE (one-way)", res_fe)



print("\n>>> MUNDLAK (RE + country means of ALL X): RandomEffects + constant")

df_m = df_levels.sort_values([ID_COL, T_COL]).copy()
for c in X_ALL:
    df_m[c + "_mean_i"] = df_m.groupby(ID_COL)[c].transform("mean")

mundlak_cols = X_ALL + [c + "_mean_i" for c in X_ALL]
X_m_raw = df_m[mundlak_cols]

X_m, vif_m = drop_high_vif(X_m_raw, VIF_CUT, verbose=True)
print("  Mundlak regressors kept:", list(X_m.columns))
print("  VIF (Mundlak):\n", vif_m)

panel_m = df_m.set_index([ID_COL, T_COL])
y_m = panel_m[DEP]
X_m_panel = sm.add_constant(panel_m[X_m.columns], has_constant="add")

re_mod = RandomEffects(y_m, X_m_panel)
re_res = re_mod.fit(cov_type="clustered", cluster_entity=True) if CLUSTER else re_mod.fit()
print(re_res.summary)



print("\n>>> TWO-WAY FE: OLS on your TWFE-transformed variables (no constant)")

needed = [DEP] + X_ALL
df_t = df_twfe[[ID_COL, T_COL] + needed].dropna().copy()


for col in needed:
    if col + "_twfe" not in df_twfe.columns:
        mi = df_t.groupby(ID_COL)[col].transform("mean")
        mt = df_t.groupby(T_COL)[col].transform("mean")
        mall = df_t[col].mean()
        df_t[col + "_twfe"] = df_t[col] - mi - mt + mall
    else:
        df_t[col + "_twfe"] = df_twfe.loc[df_t.index, col + "_twfe"]

y_twfe = df_t[DEP + "_twfe"]
X_twfe_raw = df_t[[c + "_twfe" for c in X_ALL]]

X_twfe, vif_twfe = drop_high_vif(X_twfe_raw, VIF_CUT, verbose=True)
print("  TWFE regressors kept:", list(X_twfe.columns))
print("  VIF (TWFE):\n", vif_twfe)

clusters = df_t[ID_COL] if CLUSTER else None
res_twfe = fit_ols(y_twfe, X_twfe, clusters=clusters, add_const=False)
short_print("TWO-WAY FE (TWFE transform)", res_twfe)

print("\n>>> FIRST DIFFERENCES: Δ-OLS (no constant)")

df_fd = df_levels.sort_values([ID_COL, T_COL]).copy()
for c in [DEP] + X_ALL:
    df_fd["d_" + c] = df_fd.groupby(ID_COL)[c].diff()

df_fd = df_fd.dropna(subset=["d_" + DEP] + ["d_" + c for c in X_ALL]).copy()
y_fd = df_fd["d_" + DEP]
X_fd_raw = df_fd[["d_" + c for c in X_ALL]]

X_fd, vif_fd = drop_high_vif(X_fd_raw, VIF_CUT, verbose=True)
print("  FD regressors kept:", list(X_fd.columns))
print("  VIF (FD):\n", vif_fd)

clusters = df_fd[ID_COL] if CLUSTER else None
res_fd = fit_ols(y_fd, X_fd, clusters=clusters, add_const=False)
short_print("FIRST DIFFERENCES (FD)", res_fd)

def coef_table():
    def pack_sm(res, name):
        return pd.Series(res.params, index=res.X_names, name=name)

    out = pd.concat([
        pack_sm(res_be, "BE"),
        pack_sm(res_fe, "FE_within"),
        pd.Series(re_res.params, name="Mundlak_RE"),
        pack_sm(res_twfe, "TWFE"),
        pack_sm(res_fd, "FD"),
    ], axis=1)

    return out

print("\n>>> Coefficient table:")
print(coef_table())
```

```
>>> Fisher FE test (are country effects jointly significant?)
{'F': 2.3613973272170266, 'df1': 8, 'df2': 1101, 'p': 0.016070775014142447}

>>> ANCOVA / poolability test (common slopes) one-way base
{'F': 8.212777334695192, 'df1': 48, 'df2': 1053, 'p': 1.1102230246251565e-16, 'time_fe': False}

>>> ANCOVA / poolability test (common slopes) two-way base
{'F': 25.757016923937602, 'df1': 48, 'df2': 927, 'p': 1.1102230246251565e-16, 'time_fe': True}

>>> BETWEEN (BE) regression: OLS on country means (includes constant)
    [VIF] dropping 'liquid_liabilities' (VIF=15.30 > 10.0)
  BE regressors kept: ['ln_gdppc_lag5', 'mcap_gdp', 'bank_loans_gdp', 'ln_cpi', 'ln_trade_openness']
  VIF (BE):
 ln_gdppc_lag5        2.400444
mcap_gdp             2.475139
bank_loans_gdp       1.189789
ln_cpi               1.961900
ln_trade_openness    1.568157
Name: VIF, dtype: float64

================================================================================
BETWEEN (BE)
--------------------------------------------------------------------------------
const                0.496256
ln_gdppc_lag5       -0.040350
mcap_gdp            -0.040641
bank_loans_gdp      -0.001434
ln_cpi               0.003367
ln_trade_openness    0.000969
dtype: float64

 nobs=9  R2=0.6695
================================================================================

>>> WITHIN (one-way FE): within-transformed OLS (no constant)
  FE(within) regressors kept: ['ln_gdppc_lag5_within', 'liquid_liabilities_within', 'mcap_gdp_within', 'bank_loans_gdp_within', 'ln_cpi_within', 'ln_trade_openness_within']
  VIF (FE/within):
 ln_gdppc_lag5_within         2.161711
liquid_liabilities_within    1.582851
mcap_gdp_within              1.655373
bank_loans_gdp_within        2.099611
ln_cpi_within                1.660694
ln_trade_openness_within     1.235519
Name: VIF, dtype: float64

================================================================================
WITHIN / FE (one-way)
--------------------------------------------------------------------------------
ln_gdppc_lag5_within        -0.000615
liquid_liabilities_within   -0.128656
mcap_gdp_within              0.031731
bank_loans_gdp_within       -0.009952
ln_cpi_within                0.001994
ln_trade_openness_within    -0.002723
dtype: float64

 nobs=1116  R2=0.0399
================================================================================

>>> MUNDLAK (RE + country means of ALL X): RandomEffects + constant
    [VIF] dropping 'ln_trade_openness_mean_i' (VIF=47.90 > 10.0)
    [VIF] dropping 'liquid_liabilities_mean_i' (VIF=13.02 > 10.0)
  Mundlak regressors kept: ['ln_gdppc_lag5', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp', 'ln_cpi', 'ln_trade_openness', 'ln_gdppc_lag5_mean_i', 'mcap_gdp_mean_i', 'bank_loans_gdp_mean_i', 'ln
_cpi_mean_i']
  VIF (Mundlak):
 ln_gdppc_lag5           2.184988
liquid_liabilities      2.445527
mcap_gdp                2.286675
bank_loans_gdp          4.315489
ln_cpi                  1.704548
ln_trade_openness       2.324764
ln_gdppc_lag5_mean_i    2.639788
mcap_gdp_mean_i         3.428634
bank_loans_gdp_mean_i   3.055163
ln_cpi_mean_i           2.408766
Name: VIF, dtype: float64
                        RandomEffects Estimation Summary
================================================================================
Dep. Variable:       ln_gdp_growth_5y   R-squared:                        0.0518
Estimator:              RandomEffects   R-squared (Between):              0.6372
No. Observations:                1116   R-squared (Within):               0.0383
Date:                Sat, Dec 27 2025   R-squared (Overall):              0.0518
Time:                        17:40:44   Log-likelihood                    823.86
Cov. Estimator:             Clustered
                                        F-statistic:                      6.0386
Entities:                           9   P-value                           0.0000
Avg Obs:                       124.00   Distribution:                  F(10,1105)
Min Obs:                       118.00
Max Obs:                       127.00   F-statistic (robust):          5.437e+12
                                        P-value                           0.0000
Time periods:                     127   Distribution:                  F(10,1105)
Avg Obs:                       8.7874
Min Obs:                       6.0000
Max Obs:                       9.0000

                              Parameter Estimates
========================================================================================
                      Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
----------------------------------------------------------------------------------------
const                    0.3404     0.2700     1.2607     0.2077     -0.1894      0.8703
ln_gdppc_lag5           -0.0023     0.0128    -0.1772     0.8594     -0.0273      0.0228
liquid_liabilities      -0.1133     0.0480    -2.3620     0.0183     -0.2074     -0.0192
mcap_gdp                 0.0299     0.0129     2.3204     0.0205      0.0046      0.0552
bank_loans_gdp          -0.0158     0.0504    -0.3127     0.7546     -0.1147      0.0832
ln_cpi                   0.0026     0.0005     4.8734     0.0000      0.0015      0.0036
ln_trade_openness        0.0047     0.0019     2.5080     0.0123      0.0010      0.0083
ln_gdppc_lag5_mean_i    -0.0164     0.0334    -0.4900     0.6242     -0.0820      0.0492
mcap_gdp_mean_i         -0.0242     0.0315    -0.7690     0.4420     -0.0860      0.0376
bank_loans_gdp_mean_i    0.0435     0.0415     1.0475     0.2951     -0.0380      0.1249
ln_cpi_mean_i           -0.0019     0.0018    -1.0187     0.3086     -0.0055      0.0017
========================================================================================

>>> TWO-WAY FE: OLS on your TWFE-transformed variables (no constant)
  TWFE regressors kept: ['ln_gdppc_lag5_twfe', 'liquid_liabilities_twfe', 'mcap_gdp_twfe', 'bank_loans_gdp_twfe', 'ln_cpi_twfe', 'ln_trade_openness_twfe']
  VIF (TWFE):
 ln_gdppc_lag5_twfe         1.077105
liquid_liabilities_twfe    1.506131
mcap_gdp_twfe              1.082749
bank_loans_gdp_twfe        1.406294
ln_cpi_twfe                1.386821
ln_trade_openness_twfe     1.211558
Name: VIF, dtype: float64

================================================================================
TWO-WAY FE (TWFE transform)
--------------------------------------------------------------------------------
ln_gdppc_lag5_twfe        -0.175289
liquid_liabilities_twfe   -0.116228
mcap_gdp_twfe             -0.032740
bank_loans_gdp_twfe        0.041976
ln_cpi_twfe                0.000224
ln_trade_openness_twfe     0.008031
dtype: float64

 nobs=1116  R2=0.0839
================================================================================

>>> FIRST DIFFERENCES: Δ-OLS (no constant)
  FD regressors kept: ['d_ln_gdppc_lag5', 'd_liquid_liabilities', 'd_mcap_gdp', 'd_bank_loans_gdp', 'd_ln_cpi', 'd_ln_trade_openness']
  VIF (FD):
 d_ln_gdppc_lag5         1.005319
d_liquid_liabilities    1.513046
d_mcap_gdp              1.016562
d_bank_loans_gdp        1.500274
d_ln_cpi                1.052425
d_ln_trade_openness     1.050522
Name: VIF, dtype: float64

================================================================================
FIRST DIFFERENCES (FD)
--------------------------------------------------------------------------------
d_ln_gdppc_lag5        -0.924066
d_liquid_liabilities   -0.458900
d_mcap_gdp              0.005001
d_bank_loans_gdp        0.115599
d_ln_cpi               -0.006872
d_ln_trade_openness    -0.001844
dtype: float64

 nobs=1107  R2=0.5342
================================================================================

>>> Coefficient table:
                                 BE  FE_within  Mundlak_RE      TWFE        FD
const                      0.496256        NaN    0.340436       NaN       NaN
ln_gdppc_lag5             -0.040350        NaN   -0.002259       NaN       NaN
mcap_gdp                  -0.040641        NaN    0.029933       NaN       NaN
bank_loans_gdp            -0.001434        NaN   -0.015767       NaN       NaN
ln_cpi                     0.003367        NaN    0.002565       NaN       NaN
ln_trade_openness          0.000969        NaN    0.004680       NaN       NaN
ln_gdppc_lag5_within            NaN  -0.000615         NaN       NaN       NaN
liquid_liabilities_within       NaN  -0.128656         NaN       NaN       NaN
mcap_gdp_within                 NaN   0.031731         NaN       NaN       NaN
bank_loans_gdp_within           NaN  -0.009952         NaN       NaN       NaN
ln_cpi_within                   NaN   0.001994         NaN       NaN       NaN
ln_trade_openness_within        NaN  -0.002723         NaN       NaN       NaN
liquid_liabilities              NaN        NaN   -0.113279       NaN       NaN
ln_gdppc_lag5_mean_i            NaN        NaN   -0.016380       NaN       NaN
mcap_gdp_mean_i                 NaN        NaN   -0.024222       NaN       NaN
bank_loans_gdp_mean_i           NaN        NaN    0.043466       NaN       NaN
ln_cpi_mean_i                   NaN        NaN   -0.001874       NaN       NaN
ln_gdppc_lag5_twfe              NaN        NaN         NaN -0.175289       NaN
liquid_liabilities_twfe         NaN        NaN         NaN -0.116228       NaN
mcap_gdp_twfe                   NaN        NaN         NaN -0.032740       NaN
bank_loans_gdp_twfe             NaN        NaN         NaN  0.041976       NaN
ln_cpi_twfe                     NaN        NaN         NaN  0.000224       NaN
ln_trade_openness_twfe          NaN        NaN         NaN  0.008031       NaN
d_ln_gdppc_lag5                 NaN        NaN         NaN       NaN -0.924066
d_liquid_liabilities            NaN        NaN         NaN       NaN -0.458900
d_mcap_gdp                      NaN        NaN         NaN       NaN  0.005001
d_bank_loans_gdp                NaN        NaN         NaN       NaN  0.115599
d_ln_cpi                        NaN        NaN         NaN       NaN -0.006872
d_ln_trade_openness             NaN        NaN         NaN       NaN -0.001844
```

In [6]:

```python
def compute_vif(X: pd.DataFrame) -> pd.Series:
    Xc = X.copy()
    Xc = sm.add_constant(Xc, has_constant="add")
    vifs = {}
    for i, col in enumerate(Xc.columns):
        if col == "const":
            continue
        vifs[col] = variance_inflation_factor(Xc.values, i)
    return pd.Series(vifs).sort_values(ascending=False)

def mundlak_re(
    df: pd.DataFrame,
    y_col: str,
    X_cols: list[str],
    entity_col: str,
    time_col: str,
    vif_threshold: float = 10.0,
    robust: str = "clustered"
):
    d = df.copy()
    d = d.set_index([entity_col, time_col]).sort_index()

    for x in X_cols:
        d[f"{x}_mean"] = d.groupby(level=0)[x].transform("mean")

    Z_cols = X_cols + [f"{x}_mean" for x in X_cols]

    d_model = d[[y_col] + Z_cols].dropna()
    X_for_vif = d_model[Z_cols].astype(float)
    vif = compute_vif(X_for_vif)

    kept = list(X_for_vif.columns)
    dropped = []
    while True:
        vif = compute_vif(d_model[kept].astype(float))
        max_vif = vif.max()
        if max_vif <= vif_threshold:
            break
        worst = vif.idxmax()
        kept.remove(worst)
        dropped.append((worst, float(max_vif)))
    y = d_model[y_col]
    X = sm.add_constant(d_model[kept], has_constant="add")

    re_mod = RandomEffects(y, X)
    if robust == "clustered":
        re_res = re_mod.fit(cov_type="clustered", cluster_entity=True)
    elif robust == "robust":
        re_res = re_mod.fit(cov_type="robust")
    else:
        re_res = re_mod.fit()

    return {
        "result": re_res,
        "kept": kept,
        "dropped": dropped,
        "vif_final": compute_vif(d_model[kept].astype(float)),
        "nobs": int(re_res.nobs)
    }


y_col = "ln_gdp_growth_5y"
X_cols = ["ln_gdppc_lag5","liquid_liabilities","mcap_gdp","bank_loans_gdp","ln_cpi","ln_trade_openness"]
out = mundlak_re(df_twfe, y_col, X_cols, entity_col="country", time_col="year", vif_threshold=10.0)

print("Kept:", out["kept"])
print("Dropped (var, VIF at drop time):", out["dropped"])
print(out["vif_final"])
print(out["result"].summary)
```

```
Kept: ['ln_gdppc_lag5', 'liquid_liabilities', 'mcap_gdp', 'bank_loans_gdp', 'ln_cpi', 'ln_trade_openness', 'ln_gdppc_lag5_mean', 'mcap_gdp_mean', 'bank_loans_gdp_mean', 'ln_cpi_mean']
Dropped (var, VIF at drop time): [('ln_trade_openness_mean', 49.15483502336927), ('liquid_liabilities_mean', 13.69216475847366)]
bank_loans_gdp         4.356425
mcap_gdp_mean          3.403809
bank_loans_gdp_mean    3.183421
liquid_liabilities     2.492652
ln_gdppc_lag5_mean     2.349465
mcap_gdp               2.287390
ln_trade_openness      2.265604
ln_cpi_mean            2.251461
ln_gdppc_lag5          2.184780
ln_cpi                 1.700617
dtype: float64
                        RandomEffects Estimation Summary
================================================================================
Dep. Variable:       ln_gdp_growth_5y   R-squared:                        0.0516
Estimator:              RandomEffects   R-squared (Between):              0.6272
No. Observations:                1116   R-squared (Within):               0.0383
Date:                Sat, Dec 27 2025   R-squared (Overall):              0.0516
Time:                        17:41:38   Log-likelihood                    823.73
Cov. Estimator:             Clustered
                                        F-statistic:                      6.0097
Entities:                           9   P-value                           0.0000
Avg Obs:                       124.00   Distribution:                  F(10,1105)
Min Obs:                       118.00
Max Obs:                       127.00   F-statistic (robust):         -4.046e+11
                                        P-value                           1.0000
Time periods:                     127   Distribution:                  F(10,1105)
Avg Obs:                       8.7874
Min Obs:                       6.0000
Max Obs:                       9.0000

                              Parameter Estimates
=======================================================================================
                     Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
---------------------------------------------------------------------------------------
const                   0.3747     0.2813     1.3319     0.1832     -0.1773      0.9266
ln_gdppc_lag5          -0.0024     0.0127    -0.1874     0.8514     -0.0273      0.0226
liquid_liabilities     -0.1119     0.0486    -2.3012     0.0216     -0.2073     -0.0165
mcap_gdp                0.0293     0.0128     2.2824     0.0227      0.0041      0.0545
bank_loans_gdp         -0.0146     0.0508    -0.2871     0.7741     -0.1143      0.0851
ln_cpi                  0.0026     0.0005     5.0785     0.0000      0.0016      0.0036
ln_trade_openness       0.0046     0.0019     2.4121     0.0160      0.0009      0.0084
ln_gdppc_lag5_mean     -0.0203     0.0346    -0.5868     0.5575     -0.0881      0.0476
mcap_gdp_mean          -0.0222     0.0294    -0.7575     0.4489     -0.0799      0.0354
bank_loans_gdp_mean     0.0439     0.0442     0.9924     0.3212     -0.0429      0.1307
ln_cpi_mean            -0.0017     0.0015    -1.1802     0.2382     -0.0047      0.0012
=======================================================================================
```

In [7]:

```python
X_cols = ["ln_gdppc_lag5","liquid_liabilities","mcap_gdp","bank_loans_gdp","ln_cpi","ln_trade_openness"]
y_col = "ln_gdp_growth_5y"

# 1) Check coverage by country for each required variable
req = [y_col] + X_cols
coverage = (df_twfe.groupby("country")[req]
            .apply(lambda g: g.notna().mean())
           )

# Countries with low coverage in any required variable
worst = coverage.min(axis=1).sort_values()
print("Worst countries by minimum non-missing share across required vars:")
print(worst.head(20))

# 2) Countries that survive the complete-case requirement
complete_rows = df_twfe[req].notna().all(axis=1)
survivors = df_twfe.loc[complete_rows, "country"].unique()
print("\nCountries with at least one complete row:", sorted(survivors))
print("Count:", len(survivors))

# 3) For countries that dropped out, show which variable is the blocker
dropped = sorted(set(df_twfe["country"].unique()) - set(survivors))
print("\nDropped countries:", dropped)

for c in dropped:
    g = df_twfe[df_twfe["country"] == c]
    miss_rate = 1 - g[req].notna().mean()
    print(f"\n{c} missing-rate by variable:")
    print(miss_rate.sort_values(ascending=False))
```

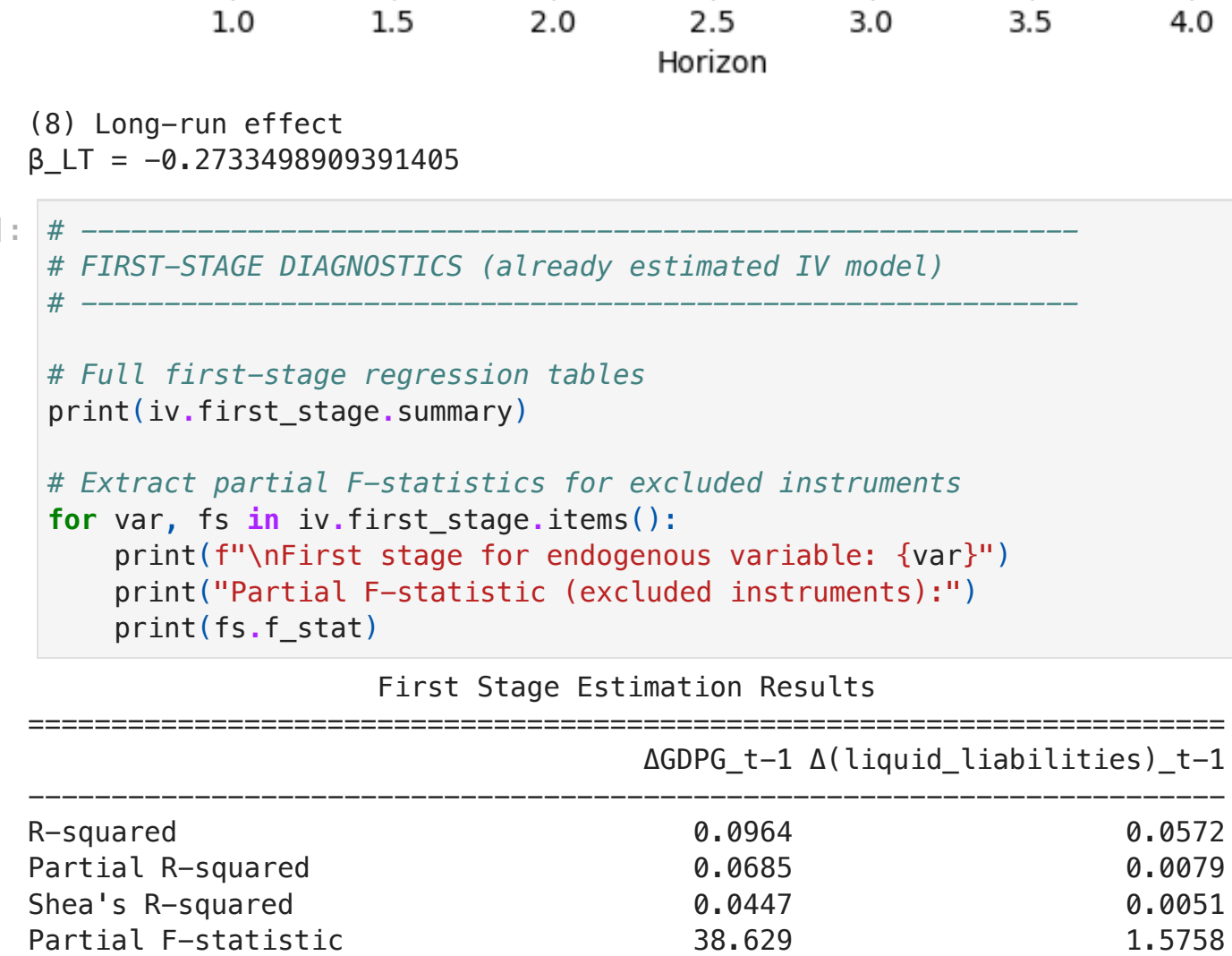
```
Worst countries by minimum non-missing share across required vars:
country
Finland           0.000000
France            0.000000
Italy             0.000000
Japan             0.000000
Netherlands       0.000000
Portugal          0.000000
Spain             0.000000
Australia         0.848921
Canada            0.848921
United States     0.848921
Denmark           0.913669
Germany           0.913669
Norway            0.913669
Sweden            0.913669
Switzerland       0.913669
United Kingdom    0.913669
dtype: float64

Countries with at least one complete row: ['Australia', 'Canada', 'Denmark', 'Germany', 'Norway', 'Sweden', 'Switzerland', 'United Kingdom', 'United States']
Count: 9

Dropped countries: ['Finland', 'France', 'Italy', 'Japan', 'Netherlands', 'Portugal', 'Spain']

Finland missing-rate by variable:
mcap_gdp              1.000000
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
bank_loans_gdp        0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

France missing-rate by variable:
bank_loans_gdp        1.000000
mcap_gdp              0.086331
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

Italy missing-rate by variable:
mcap_gdp              1.000000
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
bank_loans_gdp        0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

Japan missing-rate by variable:
mcap_gdp              1.000000
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
bank_loans_gdp        0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

Netherlands missing-rate by variable:
mcap_gdp              1.000000
bank_loans_gdp        1.000000
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

Portugal missing-rate by variable:
bank_loans_gdp        1.000000
mcap_gdp              0.086331
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
```

```
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64

Spain missing-rate by variable:
bank_loans_gdp        1.000000
mcap_gdp              0.151079
ln_gdp_growth_5y      0.035971
ln_gdppc_lag5         0.035971
liquid_liabilities    0.000000
ln_cpi                0.000000
ln_trade_openness     0.000000
dtype: float64
```

In [10]:

```python
df = df_twfe.copy()

ENTITY = "country"
TIME   = "year"


Y_raw = "ln_gdp_growth_5y"
X_raw = "liquid_liabilities"
CONTROLS_raw = ["ln_cpi", "ln_trade_openness"]


Y = "GDPG"
X = "liquid_liabilities"


rename_map = {Y_raw: Y, X_raw: X}
rename_map.update({c: c for c in CONTROLS_raw})
df = df.rename(columns=rename_map)

CONTROLS = CONTROLS_raw[:]

df = df.sort_values([ENTITY, TIME]).copy()

vars_needed = [Y, X] + CONTROLS
for v in vars_needed:
    df[v] = pd.to_numeric(df[v], errors="coerce")


df[f"{Y}_l1"] = df.groupby(ENTITY)[Y].shift(1)
df[f"{Y}_l2"] = df.groupby(ENTITY)[Y].shift(2)
df[f"{X}_l1"] = df.groupby(ENTITY)[X].shift(1)
df[f"{X}_l2"] = df.groupby(ENTITY)[X].shift(2)
df[f"d_{Y}"] = df[Y] - df[f"{Y}_l1"]
df[f"d_{X}"] = df[X] - df[f"{X}_l1"]
df[f"d_{Y}_l1"] = df.groupby(ENTITY)[f"d_{Y}"].shift(1)
df[f"d_{X}_l1"] = df.groupby(ENTITY)[f"d_{X}"].shift(1)
for c in CONTROLS:
    df[f"{c}_l1"] = df.groupby(ENTITY)[c].shift(1)
    df[f"d_{c}"] = df[c] - df[f"{c}_l1"]
needed = [
    f"d_{Y}", f"d_{Y}_l1",
    f"d_{X}", f"d_{X}_l1",
    f"{Y}_l2", f"{X}_l2"
] + [f"d_{c}" for c in CONTROLS]

df = df.dropna(subset=needed).copy()

print("Observations:", len(df))
print("Countries:", df[ENTITY].nunique())
print("\nAutocorrelation of Δy (lag 1):")
print(df.groupby(ENTITY)[f"d_{Y}"].apply(lambda s: s.autocorr(1)).describe())
def univar(s):
    return pd.Series({
        "mean": s.mean(),
        "median": s.median(),
        "std": s.std(),
        "stderr": s.std() / np.sqrt(len(s)),
        "skew": s.skew(),
        "kurt": s.kurt(),
        "min": s.min(),
        "max": s.max(),
        "n": len(s)
    })

vars_u = [f"d_{Y}", f"d_{Y}_l1", f"d_{X}", f"d_{X}_l1", f"{Y}_l2", f"{X}_l2"]
univar_table = pd.DataFrame({v: univar(df[v]) for v in vars_u}).T
print("\n(1) Univariate statistics")
print(univar_table)
corr_vars = [f"d_{Y}_l1", f"d_{X}", f"d_{X}_l1", f"{Y}_l2", f"{X}_l2"]
print("\n(2) Correlation matrix")
print(df[corr_vars].corr())
def fisher_adf(panel, var, entity_col, maxlag=1, regression="c", autolag=None, p_floor=1e-16):
    pvals = []
    used = 0
    skipped = 0

    for ent, g in panel.groupby(entity_col):
        s = g[var].dropna().values
        if len(s) <= maxlag + 2:
            skipped += 1
            continue
        try:
            res = adfuller(s, maxlag=maxlag, regression=regression, autolag=autolag)
            p = float(res[1])
            p = max(p, p_floor)  # avoid log(0)
            pvals.append(p)
            used += 1
        except Exception:
            skipped += 1
            continue

    if used == 0:
        return np.nan, np.nan, {"used": 0, "skipped": skipped}

    fisher_stat = -2.0 * np.sum(np.log(pvals))
    df_chi = 2 * used
    pval = chi2.sf(fisher_stat, df_chi)

    meta = {"used": used, "skipped": skipped, "df": df_chi, "min_p": min(pvals), "max_p": max(pvals)}
    return fisher_stat, pval, meta

print("(3) Fisher-ADF tests")
stat_y, p_y, meta_y = fisher_adf(df, f"d_{Y}", entity_col=ENTITY, maxlag=1, regression="c", autolag=None)
stat_x, p_x, meta_x = fisher_adf(df, f"d_{X}", entity_col=ENTITY, maxlag=1, regression="c", autolag=None)

print("Δy:", (stat_y, p_y), meta_y["used"], "entities used,", meta_y["skipped"], "skipped")
print("Δx:", (stat_x, p_x), meta_x["used"], "entities used,", meta_x["skipped"], "skipped")
y_ols = df[f"d_{Y}"]
X_ols = df[[f"d_{Y}_l1", f"d_{X}", f"d_{X}_l1"] + [f"d_{c}" for c in CONTROLS]].copy()
X_ols = X_ols.rename(columns={
    f"d_{Y}_l1": f"Δ{Y}_t-1",
    f"d_{X}":    f"Δ({X})_t",
    f"d_{X}_l1": f"Δ({X})_t-1",
    **{f"d_{c}": f"Δ{c}_t" for c in CONTROLS}
})

X_ols = sm.add_constant(X_ols)

ols = sm.OLS(y_ols, X_ols).fit(cov_type="HC1")

print("\n(4) OLS FD-ARDL")
print(ols.summary())
dep = df[f"d_{Y}"].copy()
exog = df[[f"d_{X}"] + [f"d_{c}" for c in CONTROLS]].copy()
exog = exog.rename(columns={
    f"d_{X}": f"Δ({X})_t",
    **{f"d_{c}": f"Δ{c}_t" for c in CONTROLS}
})
exog = sm.add_constant(exog)
endog = df[[f"d_{Y}_l1", f"d_{X}_l1"]].copy()
endog = endog.rename(columns={
    f"d_{Y}_l1": f"Δ{Y}_t-1",
    f"d_{X}_l1": f"Δ({X})_t-1"
})
instr = df[[f"{Y}_l2", f"{X}_l2"]].copy()
instr = instr.rename(columns={
    f"{Y}_l2": f"{Y}_t-2 (level)",
    f"{X}_l2": f"({X})_t-2 (level)"
})

iv = IV2SLS(dep, exog, endog, instr).fit(cov_type="robust")

print("\n(5) Anderson-Hsiao IV")
print(iv.summary)
print("\n(6) First-stage regressions")
print(iv.first_stage.summary)

# ------------------------------------------------------------
# 9. (7) IMPULSE RESPONSE FUNCTIONS (safe: use names, not positions)
#    βy = coeff on ΔGDPG_{t-1}
#    β1 = coeff on ΔX_t
#    β2 = coeff on ΔX_{t-1}
# ------------------------------------------------------------
beta_y_name = f"Δ{Y}_t-1"
beta_1_name = f"Δ({X})_t"
beta_2_name = f"Δ({X})_t-1"

βy = float(iv.params[beta_y_name])
β1 = float(iv.params[beta_1_name])
β2 = float(iv.params[beta_2_name])

IRF = [
    β1,
    βy * β1 + β2,
    (βy**2) * β1 + βy * β2,
    (βy**3) * β1 + (βy**2) * β2
]

irf_df = pd.DataFrame({"t": [1, 2, 3, 4], "IRF": IRF})
print("\n(7) IRFs")
print(irf_df)

plt.plot(irf_df["t"], irf_df["IRF"], marker="o")
plt.xlabel("Horizon")
plt.ylabel("Δ GDPG")
plt.title("Impulse Response: 1-unit increase in X")
plt.show()

β_LT = (β1 + β2) / (1 - βy)
print("\n(8) Long-run effect")
print("β_LT =", β_LT)
```

```
Observations: 2112
Countries: 16

Autocorrelation of Δy (lag 1):
count    16.000000
mean      0.207681
std       0.182037
min      -0.185585
25%       0.125790
50%       0.227598
75%       0.317863
max       0.466952
Name: d_GDPG, dtype: float64

(1) Univariate statistics
                           mean    median       std    stderr      skew  \
d_GDPG                -0.000266 -0.001509  0.074836  0.001628  0.667090   
d_GDPG_l1              0.000106 -0.001302  0.074651  0.001624  0.666517   
d_liquid_liabilities   0.005504  0.005124  0.068793  0.001497 -5.830477   
d_liquid_liabilities_l1  0.004378  0.004927  0.067442  0.001468 -6.243464   
GDPG_l2                0.096491  0.100122  0.132004  0.002872 -1.097063   
liquid_liabilities_l2  0.595839  0.551024  0.249750  0.005434  1.318177   

                             kurt       min       max       n  
d_GDPG                  19.581472 -0.766403  0.791372  2112.0  
d_GDPG_l1               19.776906 -0.766403  0.791372  2112.0  
d_liquid_liabilities   130.266391 -1.493865  0.609135  2112.0  
d_liquid_liabilities_l1  140.558559 -1.493865  0.609135  2112.0  
GDPG_l2                  9.395162 -0.946174  0.803192  2112.0  
liquid_liabilities_l2    3.603346  0.057906  2.259868  2112.0  

(2) Correlation matrix
                         d_GDPG_l1  d_liquid_liabilities  \
d_GDPG_l1                 1.000000             -0.136539   
d_liquid_liabilities     -0.136539              1.000000   
d_liquid_liabilities_l1  -0.304802              0.222596   
GDPG_l2                  -0.284774              0.171308   
liquid_liabilities_l2     0.027101             -0.126784   

                         d_liquid_liabilities_l1   GDPG_l2  \
d_GDPG_l1                              -0.304802 -0.284774   
d_liquid_liabilities                    0.222596  0.171308   
d_liquid_liabilities_l1                 1.000000  0.098385   
GDPG_l2                                 0.098385  1.000000   
liquid_liabilities_l2                  -0.095948 -0.143237   

                         liquid_liabilities_l2  
d_GDPG_l1                             0.027101  
d_liquid_liabilities                 -0.126784  
d_liquid_liabilities_l1              -0.095948  
GDPG_l2                              -0.143237  
liquid_liabilities_l2                 1.000000  
(3) Fisher-ADF tests
Δy: (np.float64(797.7323444661399), np.float64(4.865575826677976e-147)) 16 entities used, 0 skipped
Δx: (np.float64(793.0860928203259), np.float64(4.550937451556444e-146)) 16 entities used, 0 skipped

(4) OLS FD-ARDL
                            OLS Regression Results                            
==============================================================================
Dep. Variable:                 d_GDPG   R-squared:                       0.149
Model:                            OLS   Adj. R-squared:                  0.147
Method:                 Least Squares   F-statistic:                     10.10
Date:                Sat, 27 Dec 2025   Prob (F-statistic):           1.44e-09
Time:                        17:44:25   Log-Likelihood:                 2648.8
No. Observations:                2112   AIC:                            -5286.
Df Residuals:                    2106   BIC:                            -5252.
Df Model:                           5                                         
Covariance Type:                  HC1                                         
=============================================================================================
                                coef    std err          z      P>|z|      [0.025      0.975]
---------------------------------------------------------------------------------------------
const                         0.0019      0.002      1.126      0.260      -0.001       0.005
ΔGDPG_t-1                     0.2104      0.054      3.866      0.000       0.104       0.317
Δ(liquid_liabilities)_t      -0.3311      0.076     -4.339      0.000      -0.481      -0.182
Δ(liquid_liabilities)_t-1     0.0764      0.051      1.501      0.133      -0.023       0.176
Δln_cpi_t                    -0.0130      0.004     -3.385      0.001      -0.020      -0.005
Δln_trade_openness_t          0.0085      0.024      0.357      0.721      -0.038       0.055
==============================================================================
Omnibus:                      501.570   Durbin-Watson:                   1.944
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            26271.819
Skew:                           0.094   Prob(JB):                         0.00
Kurtosis:                      20.277   Cond. No.                         17.6
==============================================================================

Notes:
[1] Standard Errors are heteroscedasticity robust (HC1)

(5) Anderson-Hsiao IV
                          IV-2SLS Estimation Summary                          
==============================================================================
Dep. Variable:                 d_GDPG   R-squared:                     -1.3326
Estimator:                    IV-2SLS   Adj. R-squared:                -1.3382
No. Observations:                2112   F-statistic:                    81.960
Date:                Sat, Dec 27 2025   P-value (F-stat)                0.0000
Time:                        17:44:25   Distribution:                  chi2(5)
Cov. Estimator:                robust                                         
                                                                              
                                   Parameter Estimates                                    
==========================================================================================
                           Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
------------------------------------------------------------------------------------------
const                         0.0002     0.0035     0.0587     0.9532     -0.0066     0.0070
Δ(liquid_liabilities)_t      -0.2138     0.1498    -1.4272     0.1535     -0.5074     0.0798
Δln_cpi_t                    -0.0155     0.0028    -5.6042     0.0000     -0.0209    -0.0101
Δln_trade_openness_t         -0.0338     0.0254    -1.3304     0.1834     -0.0836     0.0160
ΔGDPG_t-1                     1.4965     0.2731     5.4796     0.0000      0.9613     2.0318
Δ(liquid_liabilities)_t-1     0.3495     0.7240     0.4828     0.6293     -1.0695     1.7685
==========================================================================================

Endogenous: ΔGDPG_t-1, Δ(liquid_liabilities)_t-1
Instruments: GDPG_t-2 (level), (liquid_liabilities)_t-2 (level)
Robust Covariance (Heteroskedastic)
Debiased: False

(6) First-stage regressions
                   First Stage Estimation Results                   
==================================================================
                                  ΔGDPG_t-1 Δ(liquid_liabilities)_t-1
------------------------------------------------------------------
R-squared                            0.0964                0.0572
Partial R-squared                    0.0685                0.0079
Shea's R-squared                     0.0447                0.0051
Partial F-statistic                  38.629                1.5758
P-value (Partial F-stat)          4.092e-09                0.4548
Partial F-stat Distn                chi2(2)               chi2(2)
================================ ===========           ===========
const                                0.0194                0.0107
                                   (3.1883)              (1.1897)
Δ(liquid_liabilities)_t             -0.0943                0.2009
                                  (-2.2878)              (2.5645)
Δln_cpi_t                            0.0011               -0.0016
                                   (0.2848)             (-0.4369)
Δln_trade_openness_t                 0.0258                0.0009
                                   (3.2732)              (0.1057)
GDPG_t-2 (level)                    -0.1497                0.0278
                                  (-6.1391)              (1.2446)
(liquid_liabilities)_t-2 (level)    -0.0075               -0.0169
                                  (-0.8619)             (-0.9056)
------------------------------------------------------------------

T-stats reported in parentheses
T-stats use same covariance type as original model

(7) IRFs
   t       IRF
0  1 -0.213792
1  2  0.029573
2  3  0.044258
3  4  0.066233
```



```
(8) Long-run effect
β_LT = -0.2733498909391405
```

In [11]:

```python
# ------------------------------------------------------------
# FIRST-STAGE DIAGNOSTICS (already estimated IV model)
# ------------------------------------------------------------

# Full first-stage regression tables
print(iv.first_stage.summary)

# Extract partial F-statistics for excluded instruments
for var, fs in iv.first_stage.items():
    print(f"\nFirst stage for endogenous variable: {var}")
    print("Partial F-statistic (excluded instruments):")
    print(fs.f_stat)
```

```
                   First Stage Estimation Results                   
==================================================================
                                  ΔGDPG_t-1 Δ(liquid_liabilities)_t-1
------------------------------------------------------------------
R-squared                            0.0964                0.0572
Partial R-squared                    0.0685                0.0079
Shea's R-squared                     0.0447                0.0051
Partial F-statistic                  38.629                1.5758
P-value (Partial F-stat)          4.092e-09                0.4548
Partial F-stat Distn                chi2(2)               chi2(2)
================================ ===========           ===========
const                                0.0194                0.0107
                                   (3.1883)              (1.1897)
Δ(liquid_liabilities)_t             -0.0943                0.2009
                                  (-2.2878)              (2.5645)
Δln_cpi_t                            0.0011               -0.0016
                                   (0.2848)             (-0.4369)
Δln_trade_openness_t                 0.0258                0.0009
                                   (3.2732)              (0.1057)
GDPG_t-2 (level)                    -0.1497                0.0278
                                  (-6.1391)              (1.2446)
(liquid_liabilities)_t-2 (level)    -0.0075               -0.0169
                                  (-0.8619)             (-0.9056)
------------------------------------------------------------------

T-stats reported in parentheses
T-stats use same covariance type as original model
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[11], line 9
      6 print(iv.first_stage.summary)
      8 # Extract partial F-statistics for excluded instruments
----> 9 for var, fs in iv.first_stage.items():
     10     print(f"\nFirst stage for endogenous variable: {var}")
     11     print("Partial F-statistic (excluded instruments):")

AttributeError: 'FirstStageResults' object has no attribute 'items'
```

In [12]:

```python
df_fe = df_twfe.copy()
df_fe = df_fe.set_index([ENTITY, TIME])

# Build lags in LEVELS
df_fe["GDPG_l1"] = df_fe.groupby(level=0)["ln_gdp_growth_5y"].shift(1)
df_fe["X_l1"]    = df_fe.groupby(level=0)["liquid_liabilities"].shift(1)

fe_vars = [
    "ln_gdp_growth_5y",
    "GDPG_l1",
    "liquid_liabilities",
    "X_l1",
    "ln_cpi",
    "ln_trade_openness"
]

df_fe = df_fe[fe_vars].dropna()

fe_model = PanelOLS(
    df_fe["ln_gdp_growth_5y"],
    sm.add_constant(df_fe[[
        "GDPG_l1",
        "liquid_liabilities",
        "X_l1",
        "ln_cpi",
        "ln_trade_openness"
    ]]),
    entity_effects=True,
    time_effects=True
)

fe_res = fe_model.fit(cov_type="robust")
print(fe_res.summary)
```

```
                          PanelOLS Estimation Summary                           
================================================================================
Dep. Variable:       ln_gdp_growth_5y   R-squared:                        0.6850
Estimator:                   PanelOLS   R-squared (Between):             -1.0889
No. Observations:                2128   R-squared (Within):               0.7279
Date:                Sat, Dec 27 2025   R-squared (Overall):              0.7021
Time:                        17:44:41   Log-likelihood                    2919.8
Cov. Estimator:                Robust                                           
                                                                                
                                        F-statistic:                      858.88
Entities:                          16   P-value                           0.0000
Avg Obs:                       133.00   Distribution:                  F(5,1975)
Min Obs:                       133.00                                           
Max Obs:                       133.00   F-statistic (robust):             220.01
                                        P-value                           0.0000
Time periods:                     133   Distribution:                  F(5,1975)
Avg Obs:                       16.000                                           
Min Obs:                       16.000                                           
Max Obs:                       16.000                                           
                                                                                
                                  Parameter Estimates                                   
======================================================================================
                    Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
--------------------------------------------------------------------------------------
const                  0.0448     0.0221     2.0278     0.0427      0.0015      0.0881
GDPG_l1                0.8202     0.0279     29.430     0.0000      0.7656      0.8749
liquid_liabilities    -0.2641     0.0701    -3.7676     0.0002     -0.4016     -0.1266
X_l1                   0.2538     0.0749     3.3870     0.0007      0.1069      0.4008
ln_cpi                 0.0005     0.0006     0.9101     0.3629     -0.0006      0.0016
ln_trade_openness      0.0050     0.0047     1.0749     0.2825     -0.0041      0.0142
======================================================================================

F-test for Poolability: 3.1379
P-value: 0.0000
Distribution: F(147,1975)

Included effects: Entity, Time
```